



MODUL PERKULIAHAN

OPTIMALISASI & OTOMASI

Kode Mata kuliah P132520004

Modul

Otomatisasi Prediktif: Random Forest,

Abstrak

Pertemuan penutup mata kuliah ini memperluas Logistic Regression baseline (Pertemuan 14) ke algoritma

Sub - CPMK

Sub-CPMK 5.2 – Mampu menginterpretasikan data kinerja dan memberikan rekomendasi perbaikan

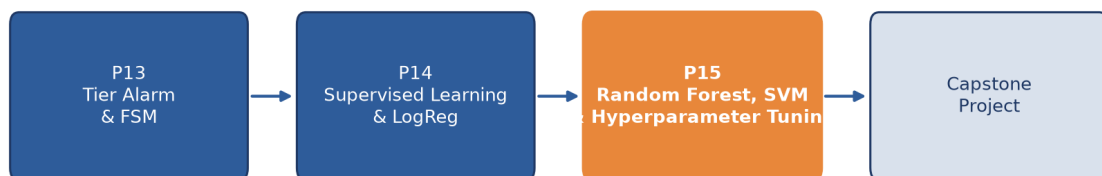
Disusun Oleh :
Dedik Romahadi, ST., M.Sc

 **Modul Interaktif:** <https://dedik-romahadi.github.io/Mechanical-Engineering-Courses/Optimalisasi-dan-Automasi/Modul/Modul-14.html>. Pada layar masuk, pilih tombol “ Mode Preview (Tanpa Login)” untuk melihat modul lengkap (animasi, kalkulator interaktif, editor kode Python langsung di browser) tanpa perlu akun. Soal pada Mode Preview bersifat tampilan saja; jawaban benar/salah dan poin tidak aktif.

PENDAHULUAN

Logistic Regression (Pertemuan 14) adalah baseline yang cepat dan interpretable, tetapi memiliki tiga keterbatasan: hanya bisa membentuk decision boundary linier, threshold 0,5 tetap belum tentu optimal, dan hasil satu kali split rawan variance tinggi. Pertemuan kelima belas, penutup mata kuliah Optimalisasi & Otomasi, memperkenalkan dua algoritma yang mengatasi keterbatasan tersebut: Random Forest dan Support Vector Machine kernel RBF, dilengkapi evaluasi lanjutan (ROC-AUC, k-fold cross-validation) dan hyperparameter tuning sistematis (GridSearchCV).

Posisi Pertemuan 15 dalam Alur Mata Kuliah — Modul Terakhir



Gambar 1. Posisi Pertemuan 15 (Random Forest, SVM & Hyperparameter Tuning) sebagai modul terakhir mata kuliah, sebelum capstone project.

Gambar 1 menunjukkan posisi Pertemuan 15 sebagai modul penutup mata kuliah.

Capaian Pembelajaran (Sub-CPMK)

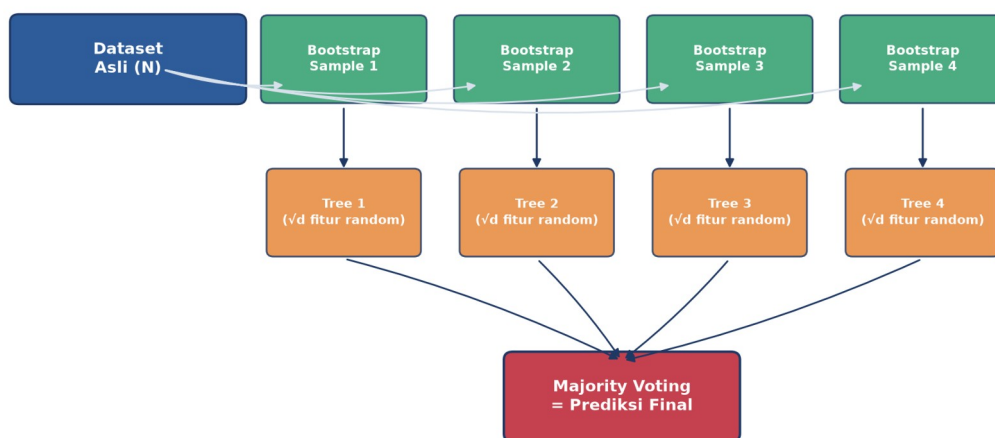
- **Sub-CPMK 5.2:** Mampu menginterpretasikan data kinerja dan memberikan rekomendasi perbaikan, yaitu menginterpretasikan metrik kinerja lanjutan (ROC-AUC, cross-validation score) untuk memberikan rekomendasi pemilihan model dan hyperparameter terbaik pada sistem klasifikasi prediktif (CPMK 5).
- Setelah pertemuan ini mahasiswa dapat: menjelaskan cara kerja Random Forest (bagging, voting) dan SVM kernel RBF; membangun Pipeline anti-leakage; menginterpretasikan kurva ROC-AUC; serta melakukan hyperparameter tuning dengan k-fold cross-validation dan GridSearchCV.

RANDOM FOREST: ENSEMBLE DECISION TREE DENGAN BOOTSTRAP + FEATURE BAGGING

Bootstrap sampling *Random feature subset ($\sqrt{n}$)* *Majority voting*
n_estimators *feature_importances_* *Out-of-bag (OOB) error*

Bagging & Random Feature Subset: Diperkenalkan Leo Breiman (2001), Random Forest membangun banyak decision tree, masing-masing dilatih pada bootstrap sample (sampling dengan pengembalian) dan hanya mempertimbangkan subset fitur acak (biasanya $\sqrt{(\text{jumlah fitur})}$) di tiap split. Prediksi akhir diambil lewat majority voting dari seluruh tree.

Random Forest: Bootstrap Aggregating + Random Feature Subset + Voting



Gambar 2. Diagram Random Forest: dataset asli menghasilkan beberapa bootstrap sample, tiap sample melatih satu tree dengan subset fitur acak, prediksi akhir via majority voting.

Gambar 2 mengilustrasikan pipeline lengkap Random Forest, dari bootstrap sampling hingga majority voting; diversifikasi antar-tree inilah yang menurunkan variance dan membuat Random Forest robust terhadap noise dibanding satu decision tree tunggal.

Out-of-Bag (OOB) Error sebagai Validasi Gratis: Out-of-bag (OOB) error menawarkan cara elegan untuk mengevaluasi performa Random Forest tanpa perlu menyisihkan data validasi terpisah: karena tiap tree hanya dilatih pada sekitar 63% data (akibat bootstrap sampling dengan pengembalian), sisa 37% data yang tidak terpakai (out-of-bag) pada tree tersebut dapat dipakai untuk mengevaluasi tree itu secara independen. Rata-rata error OOB di seluruh tree memberi estimasi generalisasi yang hampir setara dengan k-fold cross-validation, namun dihitung hampir gratis sebagai efek samping proses training itu sendiri (`RandomForestClassifier(oob_score=True)`).

Interpretasi Feature Importance: Feature importance yang dihasilkan Random Forest (`feature_importances_`) mengukur seberapa besar tiap fitur berkontribusi menurunkan impurity (Gini atau entropy) di seluruh split pada seluruh tree, dinormalisasi menjadi total

1,0. Metrik ini sangat berguna untuk interpretasi model pada konteks predictive maintenance: fitur dengan importance tertinggi biasanya mengarahkan engineer pada parameter fisik mana (RMS, kurtosis, frekuensi karakteristik bearing) yang paling diagnostik untuk kondisi kerusakan tertentu, memberi insight yang actionable, bukan sekadar prediksi black-box.

Tabel 1. Tujuh tahap workflow supervised learning (sama dengan Pertemuan 14), berlaku juga untuk Random Forest dan SVM.

Tahap Workflow	Aksi	scikit-learn API	Output
1. Load data	Read CSV/Parquet → DataFrame	pd.read_csv()	X, y arrays
2. Preprocess	Standardize, encode, impute	StandardScaler, OneHotEncoder	X_{scaled}
3. Split	Train/test 80/20 stratified	train_test_split(stratify=y)	X_{train} , X_{test} , y_{train} , y_{test}
4. Train	Fit model di train	model.fit(X_{train} , y_{train})	Trained model object
5. Evaluate	Predict + metrics di test	classification_report	Accuracy, F_1 , confusion matrix
6. Tune	Grid search hyperparams	GridSearchCV(cv=5)	Best model + params
7. Deploy	Save model, integrasi API	joblib.dump(model)	File .pkl untuk production

Tabel 1 mengingatkan kembali tujuh tahap workflow supervised learning yang berlaku untuk semua algoritma, termasuk Random Forest.

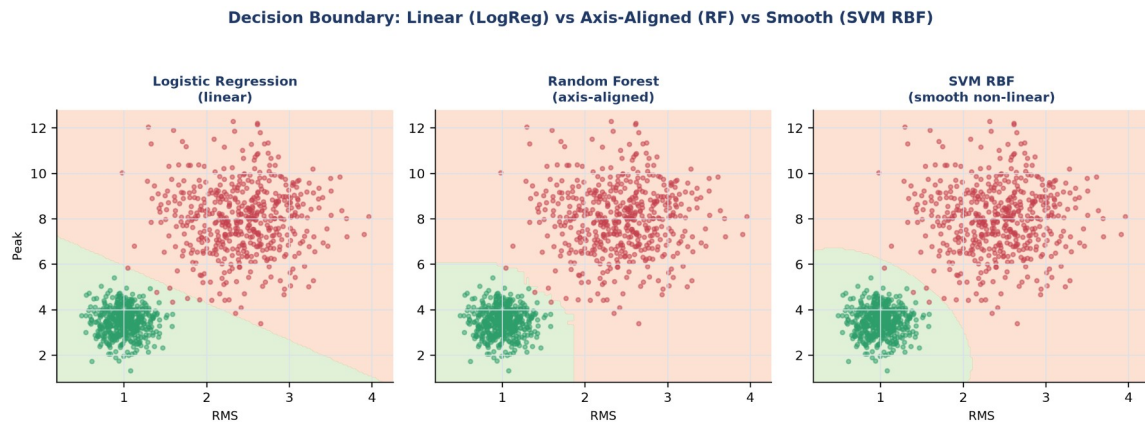
Contoh Konkret: Bearing Fault Detection (CWRU Dataset). Pada benchmark CWRU bearing dataset (4 kondisi, ~10.000 sampel/kondisi, 21 fitur), Random Forest 100 tree mencapai accuracy 98,5% dan $F_1=0,97$ (macro-average), jauh mengungguli pendekatan rule-based murni (sekitar 75%). Hubungan ini dirangkum dalam Persamaan (1).

SUPPORT VECTOR MACHINE + PIPELINE: KERNEL TRICK & ANTI-LEAKAGE WORKFLOW

Margin maximization Kernel trick: $K(x,y)=\exp(-\gamma||x-y||^2)$ (RBF) Hyperparameter C & γ StandardScaler wajib Pipeline anti-leakage (1)

Margin, Kernel Trick & Kewajiban Scaling: Diperkenalkan Vladimir Vapnik (1995), SVM mencari hyperplane pemisah dengan margin maksimum antar kelas. Untuk data yang tidak linearly separable, kernel trick (RBF, polynomial) secara implisit memetakan data ke ruang berdimensi lebih tinggi tempat batas linier menjadi cukup. SVM SANGAT sensitif terhadap skala fitur, sehingga `StandardScaler` wajib diterapkan, dan harus dibungkus dalam

‘Pipeline’ agar scaler di-fit HANYA pada data training di setiap fold cross-validation (mencegah data leakage).



Gambar 3. Perbandingan decision boundary tiga algoritma pada data yang sama: linier (Logistic Regression), axis-aligned bertingkat (Random Forest), dan smooth non-linear (SVM RBF).

Gambar 3 menunjukkan karakteristik decision boundary tiap algoritma: Logistic Regression membentuk garis lurus, Random Forest membentuk pola bertingkat axis-aligned (karena struktur pohon keputusan), sedangkan SVM RBF membentuk kurva halus non-linear yang mengikuti bentuk data secara alami.

Tabel 2. Perbandingan empat algoritma klasifikasi (diulang dari Pertemuan 14, dengan fokus baris SVM dan XGBoost).

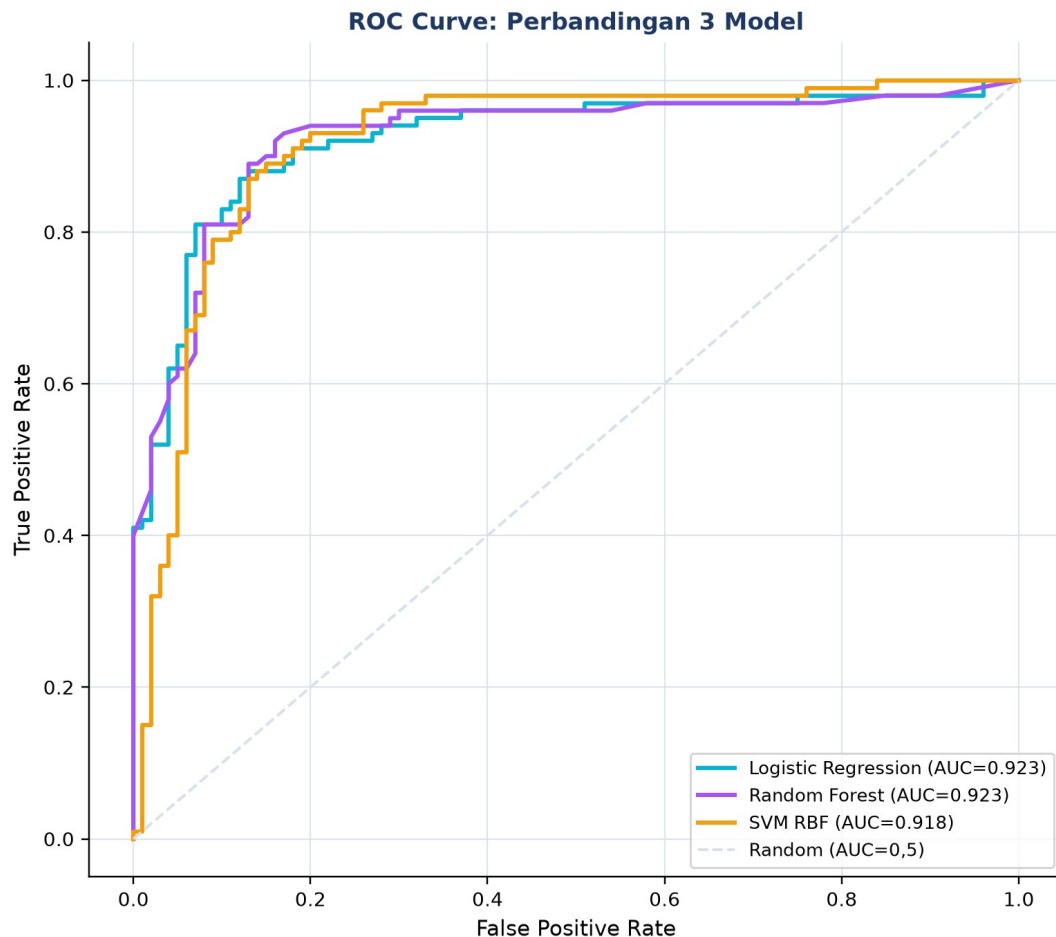
Algoritma	Train Speed	Predict Speed	Interpretability	Cocok Data
Logistic Regression	Sangat cepat	Sangat cepat	Tinggi (koefisien w)	Linear separable, <100K
Random Forest	Sedang	Cepat	Sedang (feature_imp)	Mixed, <1M, baseline terbaik
SVM (RBF)	Lambat $O(n^2)$	Sedang	Rendah	Small-medium, high-dim
XGBoost / LightGBM	Sedang	Cepat	Sedang (SHAP)	Large tabular, Kaggle-grade

Tabel 2 melengkapi perbandingan algoritma dari Pertemuan 14 dengan detail lebih pada SVM dan Gradient Boosting.

Contoh Konkret: Bearing Fault Classification (CWRU). Benchmark CWRU 4-kelas: SVM RBF ($C=10$, $\gamma=\text{scale}$) mencapai accuracy 97,2% dan $F_1=0,96$, XGBoost (default) mencapai accuracy 98,8% dan $F_1=0,98$, sedikit di atas Random Forest. Bentuk ringkasnya dituliskan pada Persamaan (2).

ROC-AUC, CROSS-VALIDATION & GRIDSEARCHCV: EVALUASI LANJUTAN & HYPERPARAMETER TUNING

ROC curve & AUC (0,5-1,0) Stratified k-fold CV: mean +/- std GridSearchCV
exhaustive class_weight='balanced' Threshold tuning cost-sensitive (2)



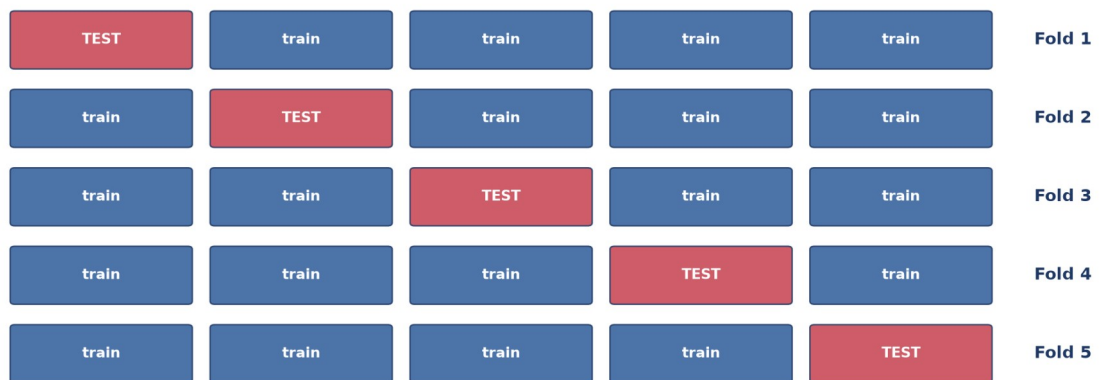
Gambar 4. Kurva ROC tiga model (Logistic Regression, Random Forest, SVM RBF) pada dataset dengan overlap kelas lebih realistis; AUC mengukur kemampuan model me-ranking sampel positif di atas negatif, independen dari threshold.

Gambar 4 menunjukkan kurva ROC ketiga model pada dataset dengan overlap kelas yang lebih realistis (bukan dataset Cell 1 yang hampir sempurna terpisah); AUC=0,5 setara tebakan acak (garis diagonal), sedangkan AUC mendekati 1,0 menandakan model yang sangat baik dalam mengurutkan sampel positif di atas negatif, terlepas dari threshold keputusan yang dipilih.

Stratified k-Fold Cross-Validation: k-fold cross-validation (biasanya k=5) membagi data training menjadi k bagian; model dilatih k kali, tiap kali menggunakan 1 fold sebagai validasi dan sisanya sebagai training, lalu skor dirata-ratakan (mean +/- std). Ini jauh lebih robust dibanding satu kali train/test split yang bisa "beruntung" atau "sial".

Mengapa Stratified, Bukan k-Fold Biasa: Stratified k-fold (bukan k-fold biasa) menjadi pilihan wajib untuk data klasifikasi imbalanced: k-fold biasa membagi data secara acak murni sehingga ada risiko satu fold kebetulan berisi sangat sedikit atau bahkan nol sampel kelas minoritas (misal fault), menghasilkan skor evaluasi yang sangat bervariasi antar-fold dan tidak dapat diandalkan. Stratified k-fold menjaga proporsi kelas pada setiap fold tetap konsisten dengan proporsi kelas pada keseluruhan dataset, sehingga estimasi performa model jauh lebih stabil dan representatif terhadap kondisi deployment sesungguhnya.

5-Fold Cross-Validation: Rotasi Test Fold, Skor Dirata-ratakan (mean $\pm$ std)

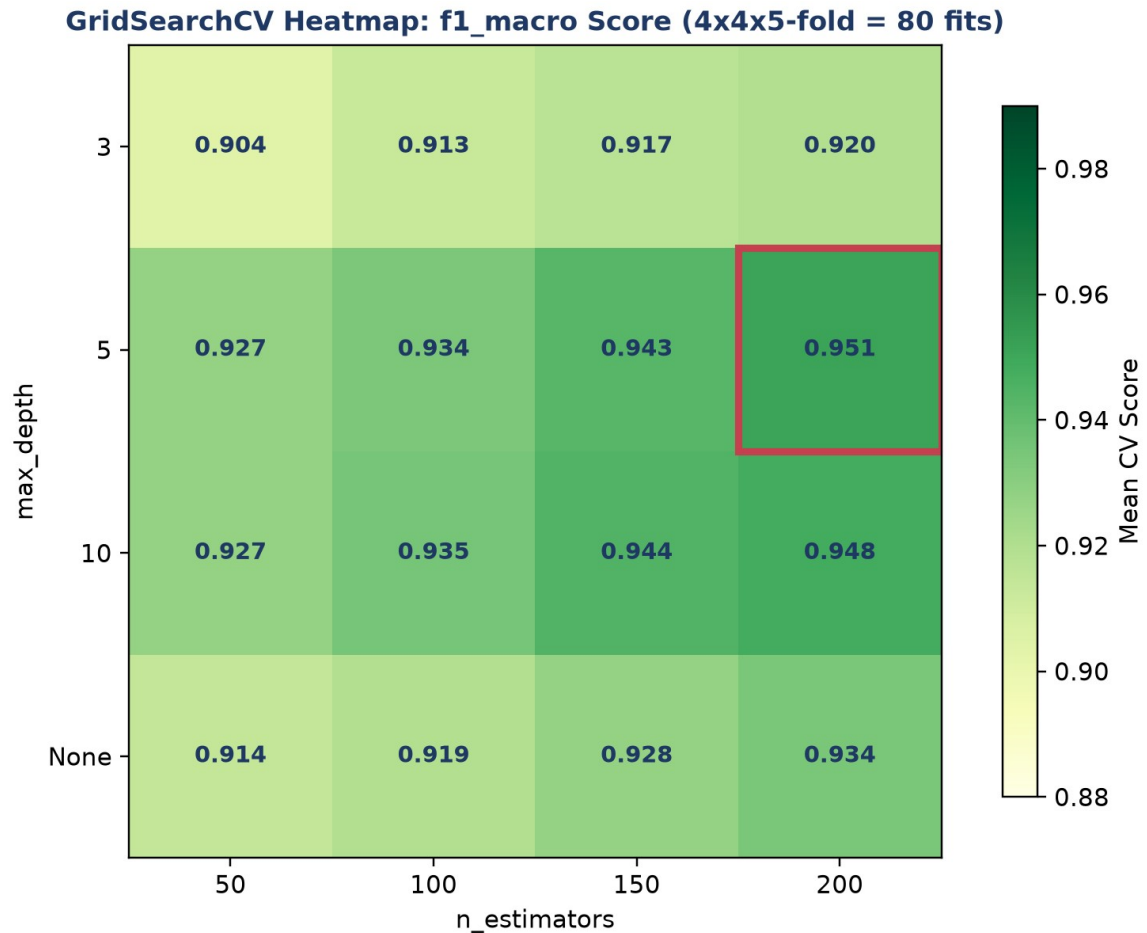


Gambar 5. 5-fold cross-validation: tiap fold bergiliran menjadi test set sedangkan 4 fold lainnya menjadi train set; skor akhir adalah rata-rata dari 5 iterasi.

Gambar 5 mengilustrasikan rotasi fold pada 5-fold cross-validation; setiap sampel data pada akhirnya pernah menjadi bagian dari test set tepat satu kali.

GridSearchCV: Exhaustive Hyperparameter Search. GridSearchCV menguji seluruh kombinasi hyperparameter dalam grid secara exhaustive, masing-masing dievaluasi dengan k-fold cross-validation. Untuk grid `n_estimators=[50,100,150,200]` x `max_depth=[3,5,10,None]` dengan `cv=5`, total dilakukan $4 \times 4 \times 5 = 80$ kali fitting model.

Skalabilitas Hyperparameter Search: Biaya komputasi GridSearchCV tumbuh secara kombinatorial terhadap jumlah hyperparameter yang di-grid-search sekaligus, sehingga pada praktiknya jarang lebih dari 2-3 hyperparameter digrid bersamaan untuk model kompleks seperti Random Forest atau SVM. Alternatif yang lebih efisien untuk ruang pencarian besar adalah RandomizedSearchCV (mengambil sampel acak dari kombinasi hyperparameter, bukan seluruhnya) atau pendekatan Bayesian optimization (Optuna, Hyperopt) yang secara adaptif mengarahkan pencarian ke region hyperparameter yang lebih menjanjikan berdasarkan hasil evaluasi sebelumnya, jauh lebih hemat komputasi dibanding exhaustive grid search pada ruang pencarian berdimensi tinggi.



Gambar 6. Heatmap GridSearchCV untuk Random Forest: kombinasi $n_estimators$ dan max_depth terbaik (kotak merah) menghasilkan skor $f1_macro$ tertinggi.

Gambar 6 menunjukkan heatmap hasil GridSearchCV; kombinasi $n_estimators=200$ dan $max_depth=5$ menghasilkan skor cross-validation tertinggi pada simulasi ini, ditandai kotak merah.

Imbalanced Data & Threshold Tuning: Untuk data yang sangat imbalanced (misal 5% fault), gunakan `class_weight='balanced'` agar model memberi bobot lebih besar pada kelas minoritas saat training, atau teknik SMOTE untuk oversampling sintesis. Threshold keputusan juga sebaiknya di-tuning (bukan default 0,5) berdasarkan biaya relatif false negative vs false positive.

Peringatan, Trap Evaluasi yang Sering Diabaikan: Tujuh jebakan evaluasi lanjutan yang sering diabaikan: memakai accuracy untuk data imbalanced; hanya mengandalkan satu kali split; data leakage (scaler di-fit sebelum split, bukan di dalam Pipeline); menggunakan ulang test set untuk tuning berulang kali; distribusi data training berbeda dari kondisi deployment (concept drift); threshold default 0,5 tanpa mempertimbangkan biaya kesalahan; serta melaporkan metrik yang tidak sesuai konteks masalah.

ANALOGI: MACHINE LEARNING DI SEHARI-HARI

- **Email Spam Filter (Binary Classification):** filter spam Gmail mencapai akurasi lebih dari 99,9% dengan menggabungkan banyak model dan aturan; klasifikasi biner spam/bukan-spam adalah contoh paling matang dari supervised learning.
- **Rekomendasi YouTube (Multi-Class + Regression):** sistem rekomendasi menggabungkan collaborative filtering dan content-based filtering, mirip bagaimana Random Forest menggabungkan banyak "opini" tree.
- **Face Detection di Kamera (Computer Vision):** Viola-Jones (2001) memakai Haar cascade dan AdaBoost (ensemble learning, sekeluarga dengan Random Forest), sedangkan sistem modern beralih ke CNN.
- **Bank Credit Scoring (Logistic Regression Klasik):** skor kredit FICO tetap memakai Logistic Regression karena regulasi menuntut interpretability tinggi, meski model yang lebih kompleks (Random Forest, gradient boosting) bisa lebih akurat.
- **Diagnosis Medis ML (Decision Tree + Ensemble):** IBM Watson Health dan Google DeepMind Health memakai ensemble XGBoost dan deep learning; hyperparameter tuning menjadi krusial karena taruhannya adalah keselamatan pasien.
- **Self-Driving Car Object Detection (Real-Time ML):** Tesla Autopilot, Waymo, dan Mobileye memakai YOLO/SSD hasil hyperparameter tuning ekstensif untuk mencapai latency di bawah 100ms tanpa mengorbankan akurasi deteksi.

Pola yang Sama di Semua Analogi: Pola yang sama di semua analogi: tidak ada satu algoritma yang selalu terbaik untuk semua kasus; pemilihan model dan hyperparameter yang tepat membutuhkan evaluasi sistematis (cross-validation) dan pertimbangan trade-off interpretability vs akurasi vs kecepatan.

APLIKASI ML PREDICTIVE MAINTENANCE DI INDUSTRI 4.0 & SMART MANUFACTURING

- **Pratt & Whitney Engine Health Management:** 5000+ mesin PW1100G/A320neo dipantau dengan ensemble RF+XGBoost dan LSTM untuk prediksi Remaining Useful Life; 80% kegagalan terdeteksi 30+ hari lebih awal, menghemat \$5-10 juta per AOG yang dihindari.
- **Siemens Mobility Train Predictive Maintenance:** platform Railigent memakai Random Forest untuk klasifikasi kondisi bogie, motor traksi, gearbox, dan rem; terbukti pada 600+ trainset dengan availability naik 15% dan MTBF naik 30%.

- **GE Predix APM:** 50.000+ aset terhubung dengan digital twin dan hybrid rule+ML untuk komponen hot gas path, menurunkan downtime 5-15%.
- **Tesla Battery Management ML:** 7104+ sel baterai per mobil dipantau dengan gradient boosting dan neural network, update model mingguan via OTA, mendukung garansi 8 tahun/240.000 km dengan penurunan kapasitas di bawah 30%.

Kaitan ke Capstone Project: Modul ini menjadi penutup mata kuliah Optimalisasi & Otomasi, menggabungkan optimasi matematis (LP/NLP/ANOVA, Pertemuan 9-11), monitoring rule-based (Pertemuan 12-13), dan machine learning (Pertemuan 14-15) menjadi satu kerangka pemikiran utuh untuk merancang sistem otomasi cerdas di industri manufaktur modern.

IMPLEMENTASI PYTHON: MACHINE LEARNING KLASIFIKASI DI JUPYTER NOTEBOOK

Empat cell berikut melanjutkan notebook Pertemuan 14, menambahkan Random Forest, SVM Pipeline, ROC-AUC, dan GridSearchCV. Lingkungan: conda env optoauto dengan paket numpy, pandas, matplotlib, scikit-learn; notebook p13_ml_classification.ipynb (lanjutan).

p13_ml_classification.ipynb — Cell 4

```
from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestClassifier

param_grid = {'n_estimators': [50,100,200],
              'max_depth': [None,5,10,20],
              'min_samples_leaf': [1,2,5]}

grid = GridSearchCV(RandomForestClassifier(random_state=42),
                    param_grid, cv=5, scoring='f1_macro', n_jobs=-1)
grid.fit(X_train, y_train)

print(f'Best params: {grid.best_params_}')
print(f'Best CV score: {grid.best_score_: .3f}')
```

Gambar 7. Potongan kode GridSearchCV untuk tuning hyperparameter Random Forest dengan 5-fold cross-validation.

Gambar 7 menunjukkan potongan kode inti Cell 4.

- **Cell 2 (lanjutan):** `RandomForestClassifier(n\_estimators=100, max\_depth=None, min\_samples\_leaf=2, random\_state=42)` dilatih pada data mentah (tanpa scaling, RF tidak butuh); cetak confusion matrix dan `feature\_importances\_` terurut.
- **Cell 3 (lanjutan):** `Pipeline([StandardScaler, SVC(C=1.0, kernel='rbf', gamma='scale', probability=True)])`; perbandingan ROC-AUC tiga model (Logistic Regression, Random Forest, SVM) via `roc\_auc\_score` dan `predict\_proba`.

- **Cell 4:** ``cross_val_score(cv=5, scoring='f1_macro')`` untuk validasi Random Forest; ``GridSearchCV`` dengan grid ``n_estimators=[50,100,200]``, ``max_depth=[None,5,10,20]``, ``min_samples_leaf=[1,2,5]`` (36 kombinasi x 5-fold=180 fit); evaluasi ``best_estimator_`` di test set; simpan model final dengan ``joblib.dump()`.`

TUGAS PERTEMUAN 15

Tugas terdiri atas 10 soal Pilihan Ganda (1 poin), 10 soal Komputasi Easy-Medium (2 poin), dan 5 soal Komputasi Hard (4 poin), total 50 poin.

Distribusi Bobot Tugas Pertemuan 15



Gambar 8. Distribusi 50 poin Tugas Pertemuan 15 berdasarkan tipe soal.

Gambar 8 merangkum distribusi bobotnya.

Bagian A: Pilihan Ganda (10 soal × 1 poin)

1. Untuk dataset PdM yang sangat imbalanced (misal 1% fault, 99% healthy), mengapa metrik accuracy menyesatkan?

- A. Accuracy selalu bernilai nol pada data imbalanced
- B. Accuracy hanya bisa dihitung untuk regresi
- C. Model yang selalu memprediksi "healthy" sudah memberi accuracy 99% padahal gagal total mendeteksi fault; pakai recall/F₁/AUC/balanced accuracy
- D. Accuracy membutuhkan GPU

2. ROC-AUC unggul sebagai metrik evaluasi karena...

- A. Bergantung pada threshold tetap 0,5
- B. Threshold-independent, mengukur kemampuan model me-ranking sampel positif di atas negatif (probabilitas fault acak lebih besar dari healthy acak). 0,5=acak, 1,0=sempurna

- C. Hanya berlaku untuk data seimbang sempurna
- D. Selalu sama dengan accuracy

3. Random Forest classifier menggabungkan banyak decision tree dengan strategi...

- A. Setiap tree dilatih di seluruh dataset yang sama, hasil dirata-rata
- B. Tiap tree dilatih di bootstrap sample dan random subset features; prediksi via majority voting. Diversifikasi mengurangi variance, robust terhadap noise
- C. Sequential boosting per tree dari error tree sebelumnya
- D. Single tree dengan max_depth tinggi

4. Support Vector Machine (SVM) dengan kernel RBF mampu menangani batas non-linear karena...

- A. Menggunakan banyak hidden layer seperti neural network
- B. Kernel trick: implicit map data ke ruang dimensi tinggi via kernel function (RBF, polynomial). Di ruang tersebut, batas linear cukup untuk memisahkan kelas yang non-linear di ruang asli
- C. Pakai gradient descent agresif
- D. Hanya bekerja untuk binary classification

5. Mengapa StandardScaler harus dibungkus dalam Pipeline bersama SVM, bukan di-fit terpisah sebelum train_test_split?

- A. Agar kode lebih pendek
- B. Mencegah data leakage: scaler harus di-fit HANYA pada data training (di tiap fold cross-validation), bukan pada seluruh dataset termasuk test set
- C. SVM tidak bisa menerima data yang sudah di-scale
- D. Pipeline mempercepat training 10x lipat

6. k-fold cross-validation (k=5) lebih reliable daripada single train/test split karena...

- A. Single split selalu underperform CV
- B. CV memberikan mean +/- std dari k metric estimates, single split bisa lucky/unlucky (variance tinggi). CV reduce variance estimate via averaging across k folds, lebih robust untuk model selection dan hyperparameter tuning
- C. CV gunakan algoritma berbeda
- D. CV lebih cepat 5 kali lipat

7. GridSearchCV dengan param_grid n_estimators:[50,100,200] dan max_depth:[None,5,10,20] serta cv=5 melakukan berapa kali fitting model?

- A. 5 kali
- B. 7 kali
- C. $3 \times 4 \times 5 = 60$ kali fitting
- D. 12 kali saja tanpa cv

8. Perbedaan utama Gradient Boosting dan Random Forest adalah...

- **A.** Keduanya identik
- **B.** RF melatih pohon secara sekuensial pada residual
- **C.** Gradient Boosting tidak memakai pohon keputusan
- **D.** RF = bagging pohon paralel (vote/average untuk mengurangi variance); Gradient Boosting = pohon sekuensial yang tiap pohon mengoreksi error pohon sebelumnya (mengurangi bias)

9. Permutation feature importance mengukur pentingnya fitur dengan cara...

- **A.** Menghapus fitur lalu melatih ulang model dari awal untuk tiap fitur
- **B.** Mengacak (shuffle) nilai satu fitur pada data test dan mengukur penurunan skor model; penurunan besar berarti fitur penting; bersifat model-agnostic
- **C.** Mengurutkan fitur secara abjad
- **D.** Hanya menghitung korelasi fitur dengan target

10. Setelah model PdM di-deploy, model drift/data drift ditangani dengan...

- **A.** Tidak perlu dipantau, model valid selamanya
- **B.** Mengganti model dengan rule-based setiap hari
- **C.** Memantau distribusi input dan metrik kinerja; bila terdeteksi drift (AUC turun, distribusi fitur bergeser), trigger retraining dengan data terbaru
- **D.** Menambah jumlah kelas target

Bagian B: Komputasi Easy-Medium (10 soal × 2 poin)

Catatan: $\text{recall (sensitivity)} = \text{TP}/(\text{TP} + \text{FN})$; $\text{specificity} = \text{TN}/(\text{TN} + \text{FP})$;
 $\text{balanced_acc} = (\text{recall} + \text{specificity})/2$; $\text{Youden J} = \text{sensitivity} + \text{specificity} - 1$; $\text{G-mean} = \sqrt{(\text{sensitivity} \times \text{specificity})}$; $\text{F-}\beta = (1 + \beta^2) \cdot \text{P.R.} / (\beta^2 \cdot \text{P} + \text{R})$; $\text{class_weight balanced: } w_{\text{kelas}} = n_{\text{samples}} / (n_{\text{classes}} \times n_{\text{kelas}})$.

C1. Confusion matrix: TP=80, FP=10, TN=85, FN=5. Hitung $\text{specificity} = \text{TN}/(\text{TN} + \text{FP})$.

-  **HINT:** Specificity = true negative rate.

C2. Dari confusion matrix yang sama (TP=80, FP=10, TN=85, FN=5), hitung $\text{balanced accuracy} = (\text{recall} + \text{specificity})/2$.

-  **HINT:** Recall = TP/(TP+FN); specificity = TN/(TN+FP); rata-ratakan.

C3. Dataset PdM: 950 sampel healthy, 50 sampel fault. Hitung $\text{rasio imbalance} = \text{mayoritas}/\text{minoritas}$.

-  **HINT:** Bagi jumlah mayoritas dengan minoritas.

C4. Untuk menyeimbangkan 50 sampel minoritas (fault) hingga sama dengan 950 mayoritas via oversampling, hitung jumlah sampel sintetis yang dibuat.

- 💡 **HINT:** Target jumlah minoritas = mayoritas; sintetis = selisihnya.

C5. Hitung Youden's $J = \text{sensitivity} + \text{specificity} - 1$ untuk $TP=80$, $FP=10$, $TN=85$, $FN=5$.

- 💡 **HINT:** $\text{sensitivity} = \text{recall}$; $J = \text{sensitivity} + \text{specificity} - 1$.

C6. Hitung F_2 -score (menekankan recall) untuk $\text{precision}=0,75$, $\text{recall}=0,60$. Formula: $F_2 = 5 \cdot P \cdot R / (4 \cdot P + R)$.

- 💡 **HINT:** Substitusi P dan R ke formula F_β dengan $\beta=2$.

C7. Skor prediksi kelas positif=[0.9, 0.4, 0.8] dan kelas negatif=[0.5, 0.3, 0.6]. Hitung ROC-AUC=fraksi pasangan (pos, neg) dengan skor pos>neg (Mann-Whitney).

- 💡 **HINT:** Bandingkan tiap skor positif dengan tiap skor negatif; hitung fraksi pos>neg.

C8. Untuk 1000 sampel, 2 kelas, minoritas 50 sampel, hitung bobot kelas minoritas ala $\text{class_weight} = \text{'balanced'} = n_{\text{samples}} / (n_{\text{classes}} \times n_{\text{minority}})$.

- 💡 **HINT:** Bagi total sampel dengan (jumlah kelas x jumlah minoritas).

C9. Biaya $FP=1$ dan biaya $FN=10$ (FN 10 kali lebih mahal). Untuk $FP=10$, $FN=5$, hitung total biaya= $FP \times 1 + FN \times 10$.

- 💡 **HINT:** Jumlahkan $FP \times \text{biaya_FP} + FN \times \text{biaya_FN}$.

C10. Hitung G-mean= $\sqrt{(\text{sensitivity} \times \text{specificity})}$ untuk $TP=80$, $FP=10$, $TN=85$, $FN=5$.

- 💡 **HINT:** Akar dari ($\text{recall} \times \text{specificity}$).

Bagian C: Komputasi Hard (5 soal × 4 poin)

Setiap soal membutuhkan 20-50+ baris kode dengan algoritma lengkap dari awal. Hint minimal. Jawaban benar: +4 poin. Jawaban salah (kode ada tapi hasil tidak sesuai/error): +1 poin partial. Kode kosong = 0 poin (bisa retry).

C11 (Hard). ROC-AUC via Mann-Whitney: `np.random.seed(42); skor_positif=np.random.normal(0.7,0.15,100); skor_negatif=np.random.normal(0.4,0.15,100)`. Hitung AUC=rata-rata atas seluruh pasangan (pos,neg): +1 bila pos>neg, +0,5 bila sama.

- 💡 **HINT:** Bandingkan tiap skor positif dengan tiap negatif (100x100 pasangan); rata-ratakan.

C12 (Hard). Threshold tuning (Youden): dengan skor pos/neg dari soal sebelumnya (seed 42), sapu semua threshold unik; di tiap threshold hitung TPR dan FPR; print nilai Youden's J maksimum= $\max(\text{TPR}-\text{FPR})$.

- 💡 **HINT:** Untuk tiap threshold t , prediksi positif bila skor $\geq t$; hitung TPR-FPR; ambil maksimum.

C13 (Hard). SMOTE 50%: 920 sampel mayoritas, 80 minoritas. Oversample minoritas hingga mencapai 50% jumlah mayoritas. Hitung jumlah sampel sintetis yang ditambahkan.

- 💡 **HINT:** Target minoritas=0,5 x mayoritas; sintetis=target-minoritas awal.

C14 (Hard). Precision pada recall tinggi: dengan skor dan label dari soal ROC-AUC (positif=label 1, negatif=label 0, seed 42), urutkan skor menurun, lalu hitung precision pada titik pertama dimana $\text{recall} \geq 0,9$.

- 💡 **HINT:** Urut skor menurun, akumulasi TP/FP, cari recall pertama $\geq 0,9$, ambil precision di titik itu.

C15 (Hard). Optimasi threshold untuk F1: dengan data seed 42 dari soal ROC-AUC, sapu semua threshold; hitung F1 di tiap threshold; print F1 maksimum.

- 💡 **HINT:** Untuk tiap threshold hitung precision, recall, lalu F_1 ; ambil maksimum.

DAFTAR PUSTAKA

- Altaf, M., Akram, T., Khan, M. A., Iqbal, M., Ch, M. M. I., & Hsu, C.-H. (2022). A new statistical features based approach for bearing fault diagnosis using vibration signals. *Sensors*, 22(5), 2012. <https://doi.org/10.3390/s22052012>
- Zhang, M., Guo, J., Li, X., & Jin, R. (2020). Data-driven anomaly detection approach for time-series streaming data. *Sensors*, 20(19), 5646. <https://doi.org/10.3390/s20195646>
- Ul Hassan, I., Panduru, K., & Walsh, J. (2024). An in-depth study of vibration sensors for condition monitoring. *Sensors*, 24(3), 740. <https://doi.org/10.3390/s24030740>
- Romanssini, M., de Aguirre, P. C. C., Compassi-Severo, L., & Girardi, A. G. (2023). A review on vibration monitoring techniques for predictive maintenance of rotating machinery. *Eng*, 4(3), 1797–1817. <https://doi.org/10.3390/eng4030102>
- Tiboni, M., Remino, C., Bussola, R., & Amici, C. (2022). A review on vibration-based condition monitoring of rotating machinery. *Applied Sciences*, 12(3), 972. <https://doi.org/10.3390/app12030972>
- Nguyen, T. T., Phi, B. P., Tran, V. T., Nguyen, V.-T., & Nguyen, V. T. T. (2025). Three-stage optimization of surface finish in WEDM of D2 tool steel via Taguchi design and ANOVA analysis. *Metals*, 15(6), 682. <https://doi.org/10.3390/met15060682>